

Analysis of Attention Mechanisms in the Context of Computational Paradigms

Experiment Report — Findings & Figures

Uroš Savurdić

School of Electrical Engineering, University of Belgrade

SICAAI 2026 — Kragujevac, May 20-21, 2026

17 paradigms benchmarked | Real hardware: CPU, NVIDIA T4, TPU v5e-1
FP32 + FP16 precision | Simulated: optical, quantum, chemical,
biological, dataflow, WSN, IoT edge, GaAs, CdTe
12 conclusions | 8 figures

Experiment Setup

17 paradigms benchmarked on the same attention mechanism computation:

Real hardware (measured on Google Colab):

- CPU (Decomposed + Fused SDPA) — Google Colab, AVX2, 4 cores
- GPU FP32 (Decomposed + Fused SDPA) — NVIDIA T4, cuBLAS
- GPU FP16 (Decomposed + Fused SDPA) — NVIDIA T4, Tensor Cores
- TPU (Decomposed + Fused) — TPU v5e-1, JAX, float32 precision

Simulated architectures (analytical models):

- Dataflow (Maxeler-style) — 4 parallel pipes, 500 MHz DFE
- Optical (Photonic MZI Mesh) — 64×64 Mach-Zehnder interferometer array
- IoT Edge (ARM Cortex-M4F) — 168 MHz, 256 KB SRAM, FPU
- GaAs RISC-V — 1 GHz, 11-instruction ISA, scalar
- WSN (Wireless Sensor Network) — 64 nodes, IEEE 802.15.4 (250 kbps)
- CdTe Turing Machine — 10 MHz, 4-operation TM
- Quantum (Gate-based) — 1000 qubits, 50 ns gate time
- Chemical (CRN) — 256-well microfluidic, 1 kHz reaction rate
- Biological (DNA) — strand displacement cascades, 1024 parallel strands

Sequence lengths: 128, 256, 512, 1024, 2048

Embedding dimensions: 64, 128, 256

Attention heads: 8

All paradigms compute the same $\text{Attention}(Q,K,V) = \text{Softmax}(QK^T/\sqrt{d}) \cdot V$ formula with identical FLOP counts. Only the hardware and execution strategy differ.

1. Architecture Determines Performance — 12+ Orders of Magnitude

The same attention computation (5.39 GFLOP at $n=2048$, $d=256$) spans from $\sim 10^{-8}$ GFLOP/s (Biological DNA computing) to 15,772 GFLOP/s (TPU Fused JAX). This trillion-fold difference comes entirely from architectural choices — parallelism, specialization, and data movement — not from the algorithm.

Even among electronic paradigms: CdTe Turing (0.0007 GFLOP/s) to TPU Fused (15,772 GFLOP/s) spans 7 orders of magnitude. Same algorithm, same FLOPs, same mathematical result. Architecture is everything.

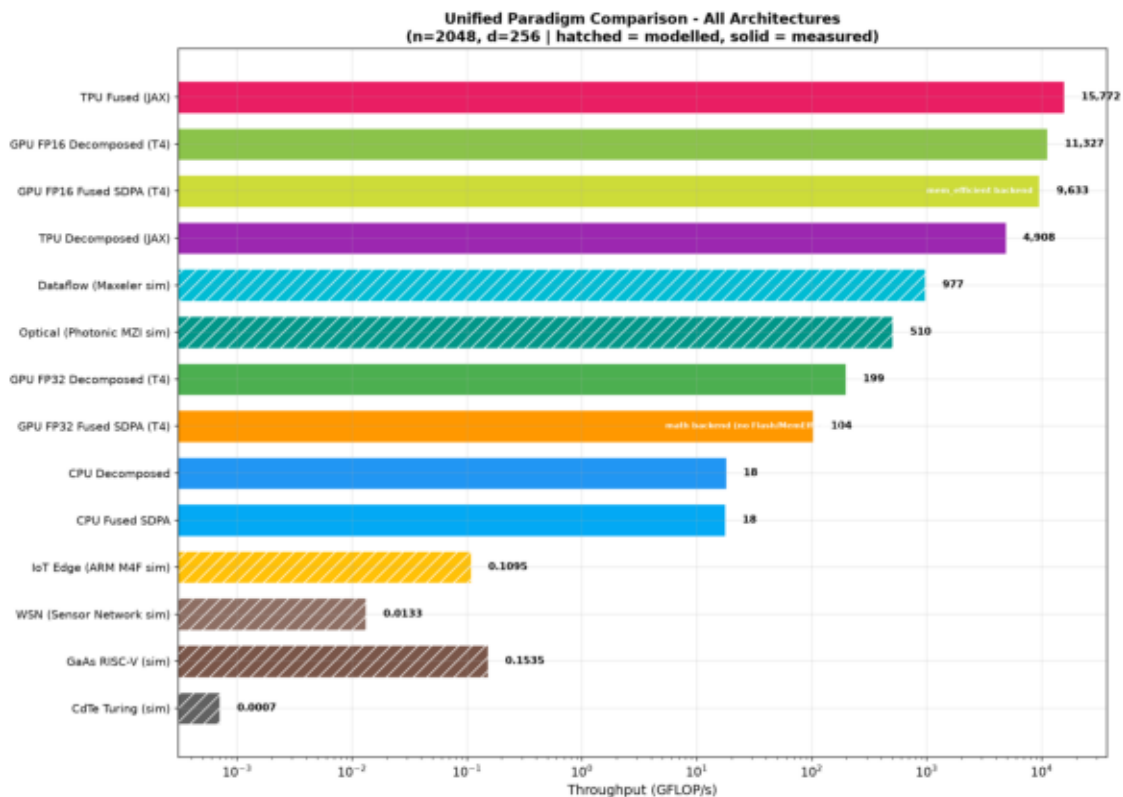


Figure 1: Unified paradigm comparison — all 17 architectures ($n=2048$, $d=256$)

2. Operator Fusion Is Hardware-Dependent, Not Universal

- TPU: Fused (15,772 GFLOP/s) beats decomposed (4,908) by 3.2x — XLA compiler eliminates intermediate memory traffic across operation boundaries.
- GPU FP32: Fused (104 GFLOP/s) is 1.9x SLOWER than decomposed (199) — FP32 on T4 forces SDPA to the naive "math" backend with no optimization.
- GPU FP16: Fused (9,633) is 1.2x slower than decomposed (11,327) — multi-head partitioning (head_dim=32) creates smaller matrices than single-head path.
- CPU: No meaningful difference (1.0x) — SDPA has no specialized backend.

Fusion is not automatically better. It depends on whether the hardware and precision support the optimized execution path.

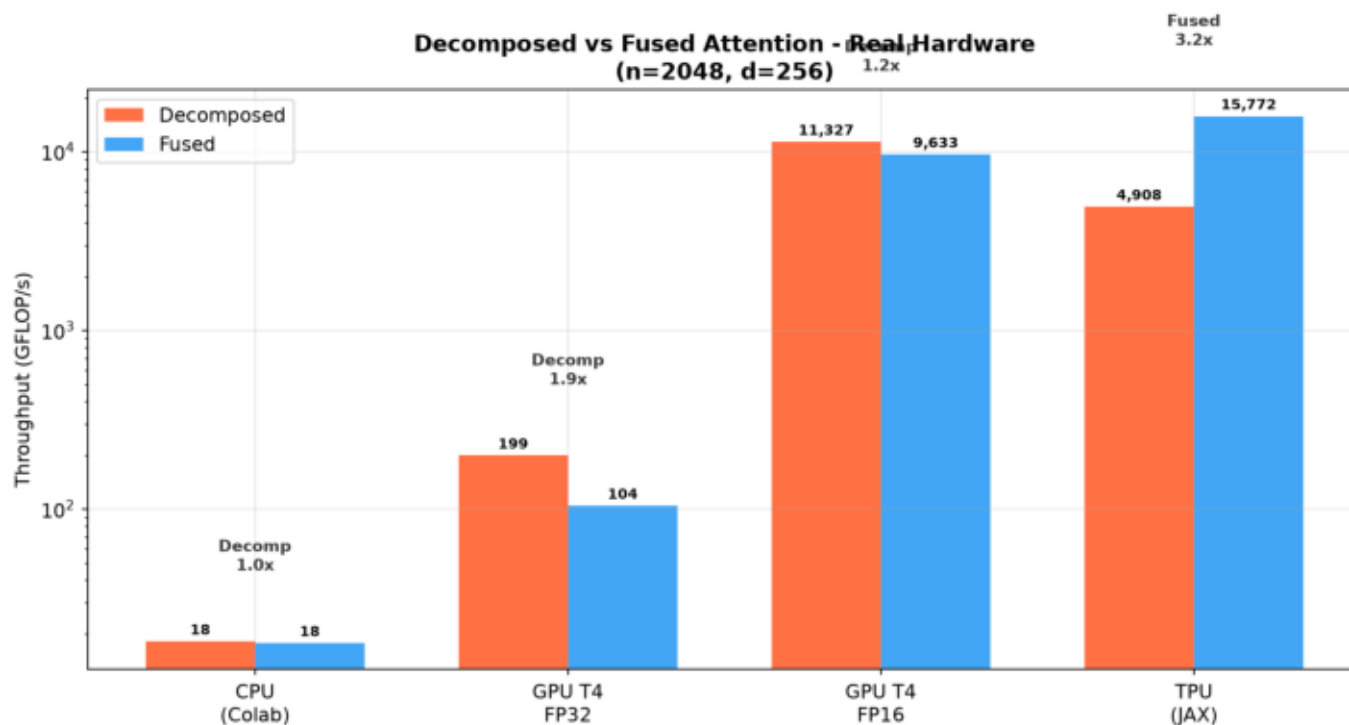


Figure 3: Fused vs decomposed attention — real hardware comparison

3. Precision and Backend Co-Design Dominate GPU Performance

On NVIDIA T4 (Turing, sm_75):

- FP32 → FP16 decomposed: 56.9x speedup (199 → 11,327 GFLOP/s) via Tensor Cores
- FP32 → FP16 fused SDPA: 92x speedup (104 → 9,633 GFLOP/s) via Tensor Cores
 - + mem_efficient backend replacing math fallback

The same one line of PyTorch code — `F.scaled_dot_product_attention(Q, K, V)` — dispatches to completely different execution strategies depending on precision and GPU generation. We verified: "math (fallback)" for FP32, "mem_efficient" for FP16. The API abstraction hides a 92x performance cliff.

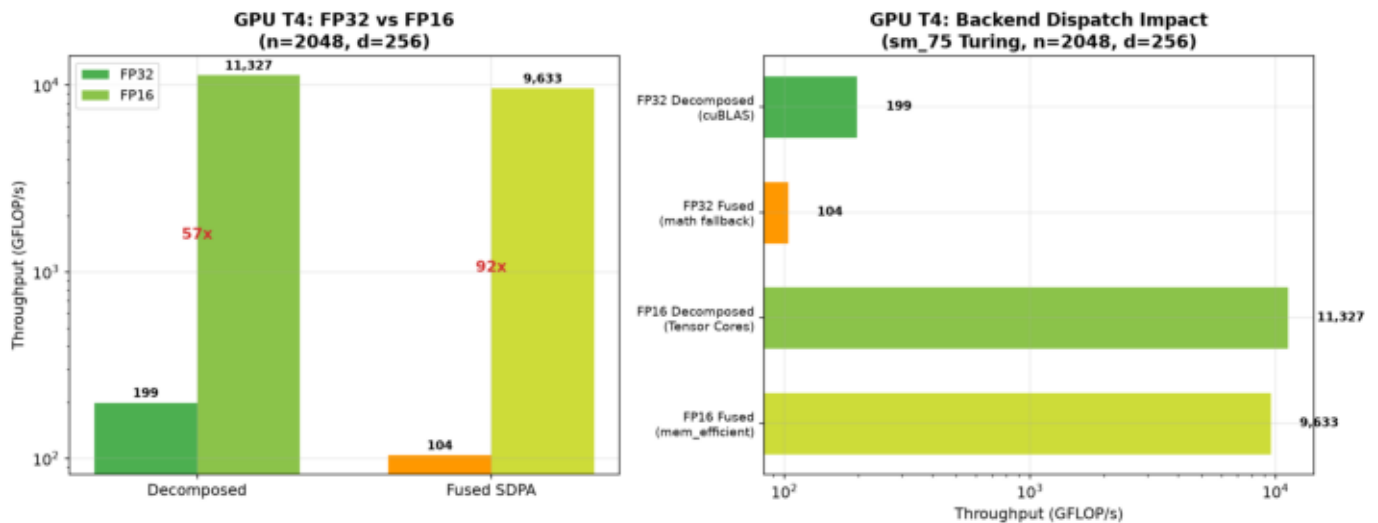


Figure 2: GPU T4 — FP32 vs FP16 precision & backend dispatch

4. Softmax Is the Architectural Discriminator

Despite comprising only ~0.4% of total FLOPs, Softmax execution time varies wildly across architectures:

- CPU: 34% of execution time — memory-bound, cache thrashing on $n \times n$ matrix
- Dataflow: 38% — pipeline stall (5 special stages vs d MAC stages for matmul)
- Optical: dominant — OEO conversion bottleneck (see Conclusion 11)
- GPU FP16: 23% — Tensor Cores accelerate matmul but not Softmax
- GPU FP32: 7% — warp-level parallel reduction in shared memory
- TPU Systolic: 21% — runs on narrow 128-wide vector unit, not systolic array
- GaAs / CdTe: 0.5% — purely FLOP-proportional (no memory effects)

The 0.4% operation that takes 0.5% to 34% of the time. Softmax reveals whether your hardware handles irregular computation well.

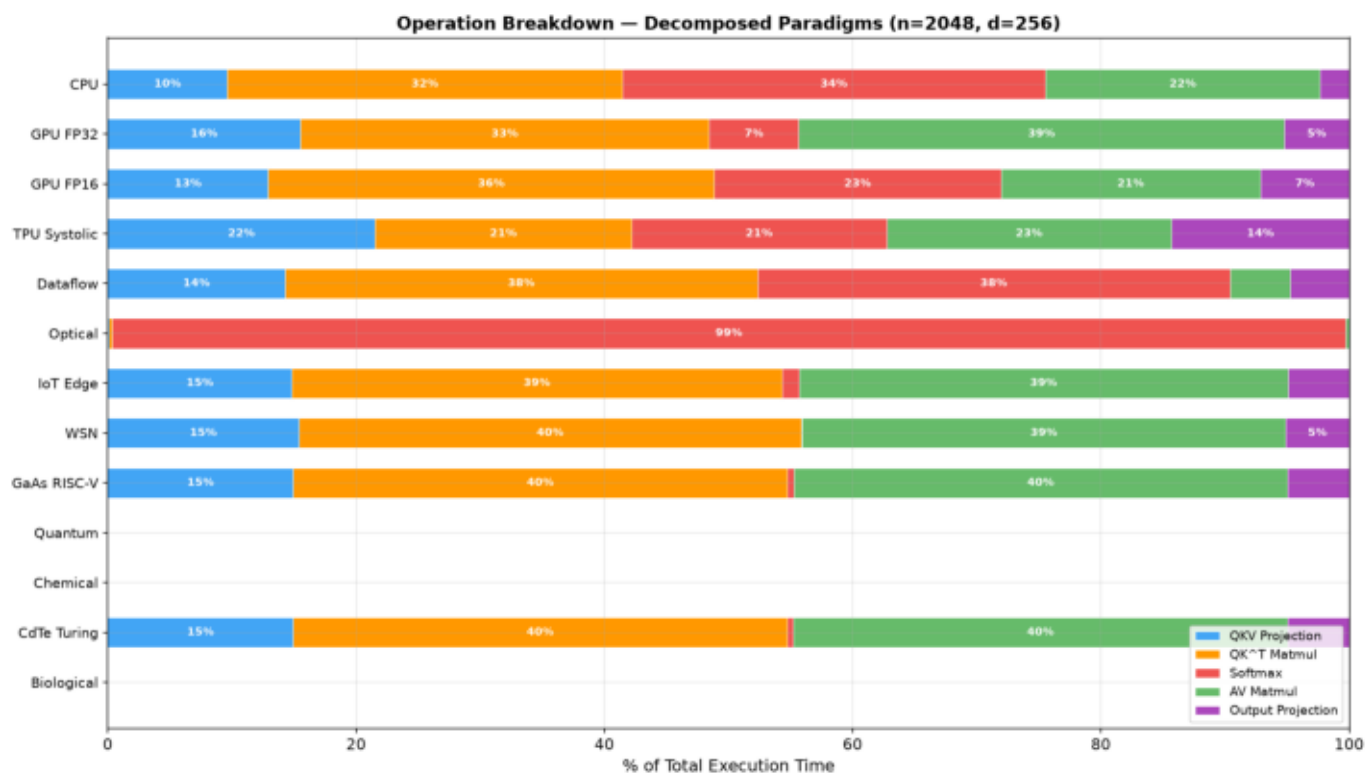


Figure 7: Operation time breakdown — decomposed paradigms (n=2048, d=256)

5. Hardware Multipliers Are Transformative

- Biological DNA: $\sim 10^{-8}$ GFLOP/s — strand displacement cascades, 10s per multiply
- Chemical CRN: 0.0002 GFLOP/s — bimolecular reactions, ms timescale
- Quantum: 0.0006 GFLOP/s — 1792 T-gates per multiply at 50 ns each
- CdTe Turing: 0.0007 GFLOP/s — every multiply is ~ 25 repeated additions
- GaAs RISC-V: 0.15 GFLOP/s — hardware FMUL (3 cycles) but no parallelism
- IoT Edge: 0.11 GFLOP/s — ARM M4F FPU, 168 MHz, tiled into 256 KB SRAM
- CPU + SIMD: 18 GFLOP/s — hardware multiplier + AVX2 parallelism
- TPU Fused: 15,772 GFLOP/s — systolic array + XLA compiler fusion

A hardware FMUL replaces 25+ primitive operations. Adding parallelism creates another 100,000x. The transistor cost is repaid billions of times.

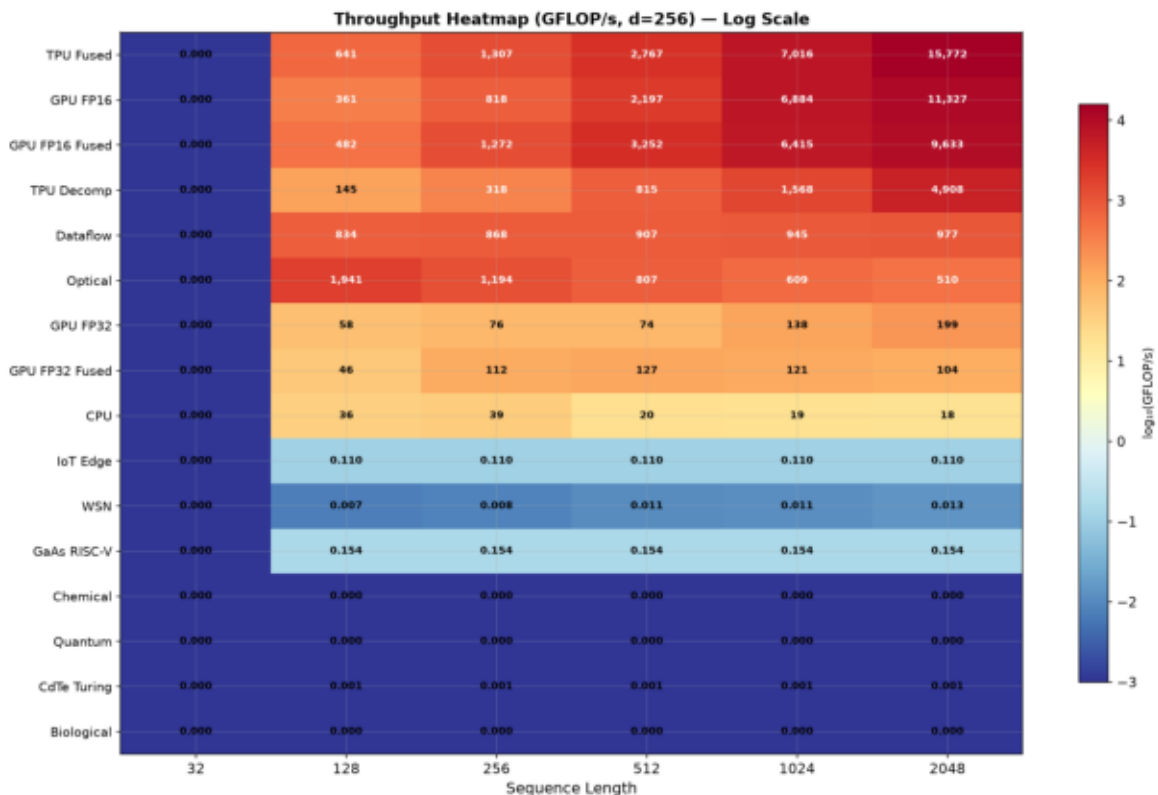


Figure 5: Throughput heatmap — paradigm × sequence length (d=256)

6. Compiler/Runtime/Hardware Co-Design Is the Real Story

The strongest practical finding: modern AI performance is not determined by any single factor but by the co-design of:

1. Hardware (systolic arrays, Tensor Cores, vector units)
2. Compiler (XLA fusion, cuBLAS kernel selection)
3. Runtime (SDPA backend dispatch, precision-aware routing)
4. Algorithm structure (single-head vs multi-head, fused vs decomposed)

The GPU FP32 fused anomaly (slower than decomposed) is not a bug — it is a systems insight showing how a mismatch at any layer of this stack can negate architectural advantages.

Performance is not a property of the hardware alone. It emerges from the alignment of hardware, compiler, runtime, and algorithm structure.

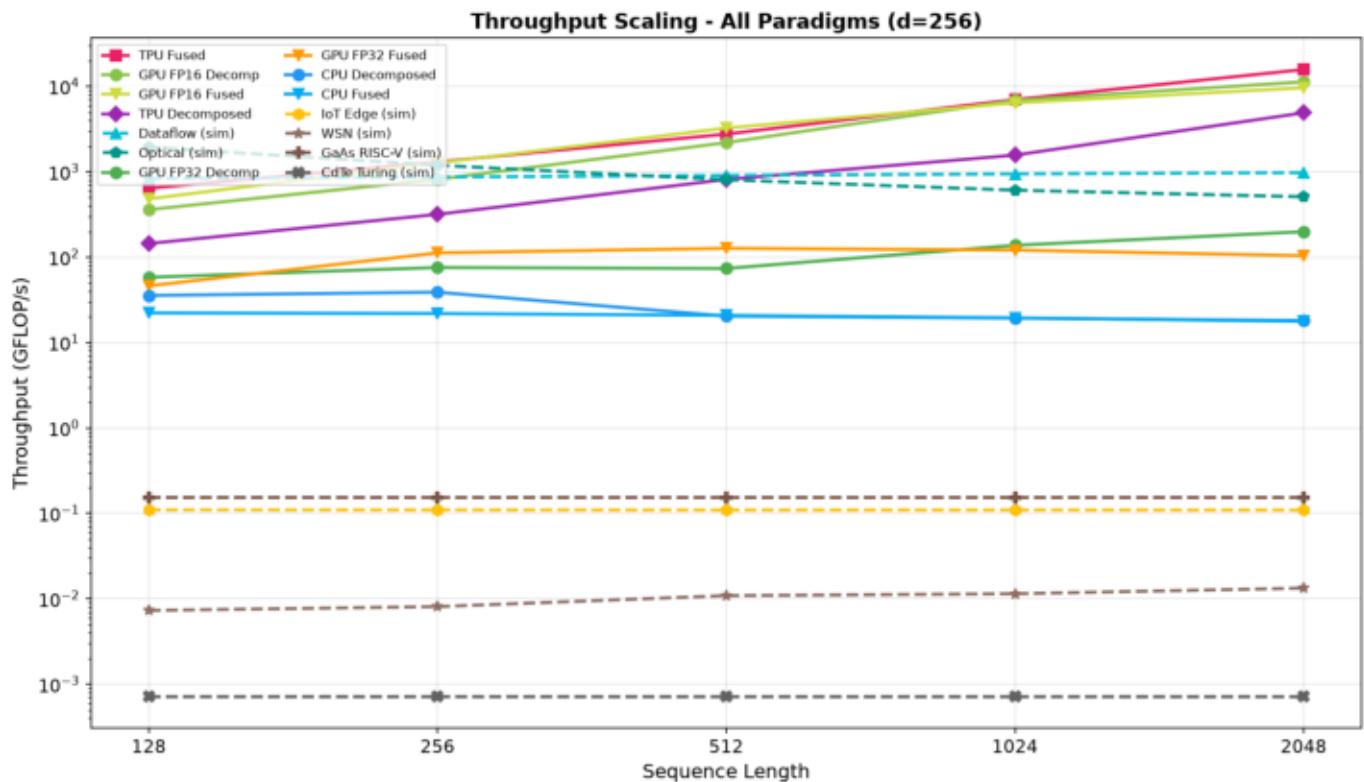


Figure 4: Throughput scaling — all paradigms ($d=256$)

7. Scaling Efficiency — Accelerators Absorb Quadratic Growth

From $n=128$ to $n=2048$ at $d=256$, total FLOPs grow 64x. But execution time growth tells a very different story:

- CPU: 126x time growth ($0.0024s \rightarrow 0.297s$) — cache pressure erodes throughput
- GPU FP16: 2.0x time growth ($0.00023s \rightarrow 0.00048s$) — Tensor Cores absorb the quadratic cost through massive parallelism
- TPU Fused: 2.6x time growth ($0.00013s \rightarrow 0.00034s$) — XLA fusion keeps the systolic array fed efficiently

The algorithm is $O(n^2d)$, but at these sizes the accelerators are latency-bound, so most of the added work is free. FLOPs grow 64 times — TPU time grows 2.6.

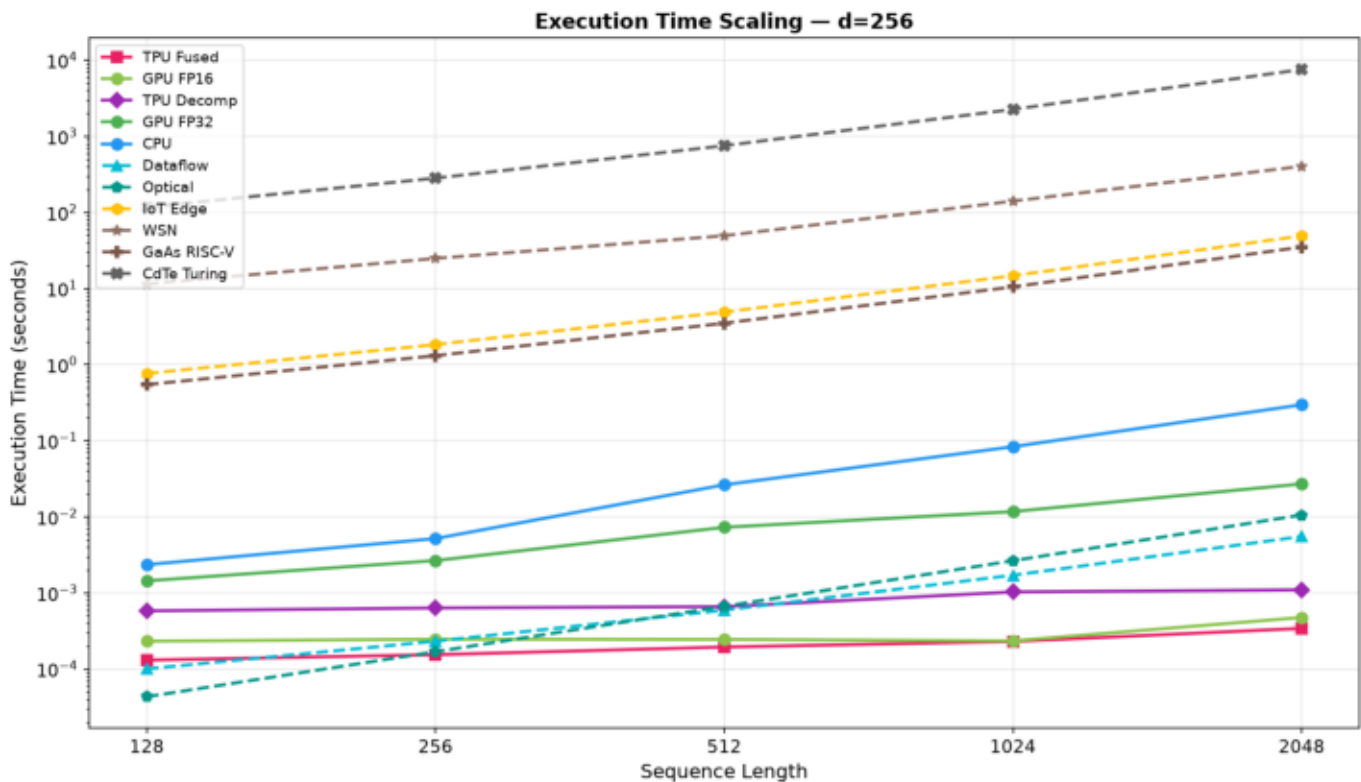


Figure 8: Execution time scaling — key paradigms ($d=256$)

8. Embedding Dimension Unlocks Hardware Utilization

Larger embedding dimensions dramatically improve accelerator throughput at $n=2048$:

- TPU Fused: 4,855 GFLOP/s ($d=64$) → 15,772 GFLOP/s ($d=256$) = 3.25x improvement
- GPU FP16: 2,728 → 11,327 GFLOP/s = 4.15x improvement
- CPU: 5.9 → 18.1 GFLOP/s = 3.06x improvement

Larger d means larger matrices, which fill systolic arrays and warp schedulers more completely. At $d=64$, the 128×128 TPU MXU is heavily underutilized. At $d=256$, tiles align and utilization approaches theoretical peak.

Bigger models aren't just more accurate — they're more efficient. Doubling the embedding dimension more than doubles throughput on accelerators.

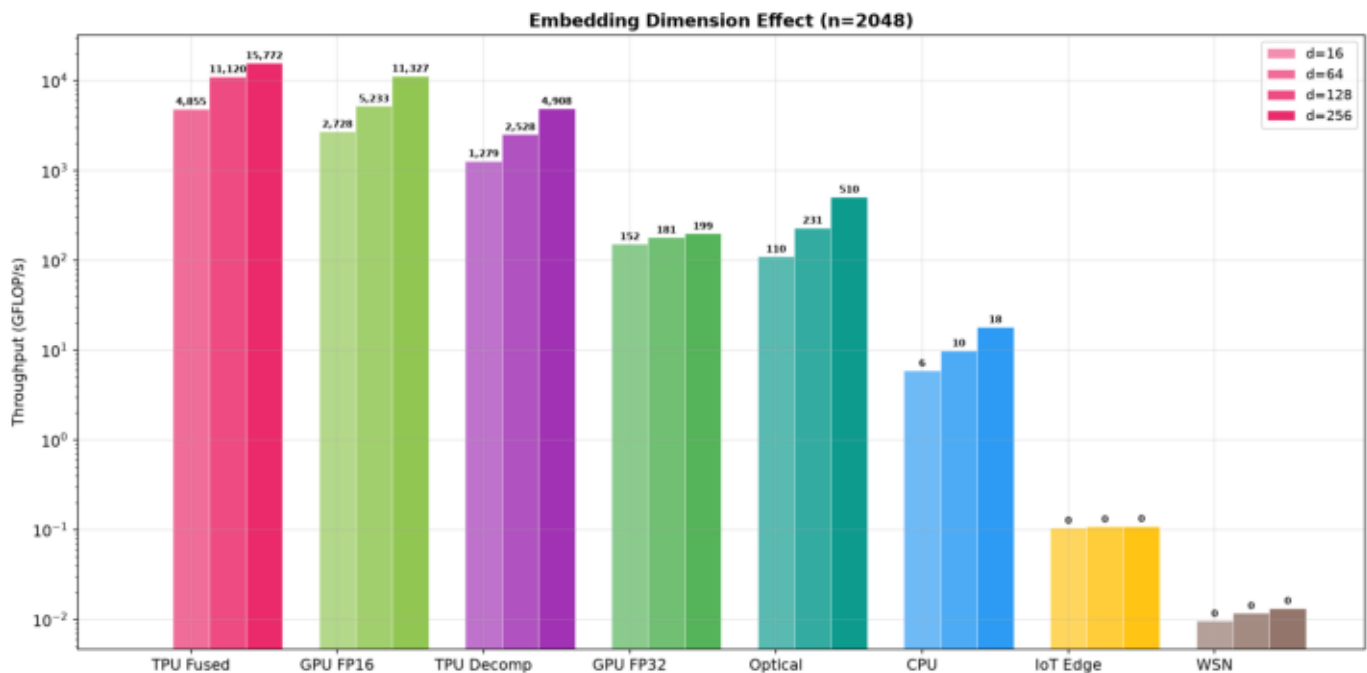


Figure 6: Embedding dimension effect on throughput ($n=2048$)

9. Scalar Architectures Reveal the Pure Algorithmic Bottleneck Problem

GaAs RISC-V and CdTe Turing Machine show identical operation time distributions: QK^T matmul 40%, AV matmul 40%, QKV projection 15%, output projection 5%, and Softmax at only 0.5%.

This is because both are sequential scalar processors with no parallelism, no cache, and no memory hierarchy — execution time tracks FLOPs directly. This makes them the algorithmic control group. The ~0.5% Softmax time reflects its true FLOP share (21M out of 5.39G total FLOPs, or 0.39%).

Compare this to the same operation on other architectures:

- CPU: 34% — memory-bound, cache thrashing on the $n \times n$ matrix
- Dataflow: 38% — pipeline stall (5 special stages vs d MAC stages)
- GPU FP32: 7% — warp-level parallel reduction in shared memory
- GaAs/CdTe: 0.5% — purely FLOP-proportional, no memory effects

The same 0.4%-of-FLOPs operation ranges from 0.5% to 38% of execution time depending on architecture. This confirms that Softmax is purely an architectural discriminator — its execution cost reveals the memory hierarchy, parallelism structure, and pipeline design of each platform rather than reflecting algorithmic complexity.

GaAs and CdTe show what the pure algorithm looks like — Softmax is 0.5% of the work. The fact that it becomes 34% on CPU tells you everything about how memory hierarchy reshapes execution.

10. Unconventional Computing Faces the Dense Matrix Wall

Four non-electronic paradigms reveal that attention is a worst-case workload for unconventional computing:

- Optical (MZI mesh): 510 GFLOP/s — photonic matmul is genuinely fast (ns-scale through 64×64 interferometer mesh), but OEO conversion for softmax creates the bottleneck. Without softmax, optical would rival GPU.
- Quantum (gate-based): 0.0006 GFLOP/s — each multiply requires 1,792 T-gates (16^2 Toffolis $\times$ 7 T-gates each) at 50 ns per gate. Quantum advantage exists for search/factoring, not for dense linear algebra.
- Chemical (CRN): 0.0002 GFLOP/s — bimolecular reactions at 1 kHz with 256-well parallelism. Each dot product requires K sequential reaction rounds.
- Biological (DNA): $\sim 10^{-8}$ GFLOP/s — strand displacement takes 10 seconds per operation with cascade depth of 6 per multiply. 11,400 seconds for $n=32$.

These paradigms are optimized for fundamentally different problem classes. Attention is pure dense linear algebra — exactly what they were not designed for.

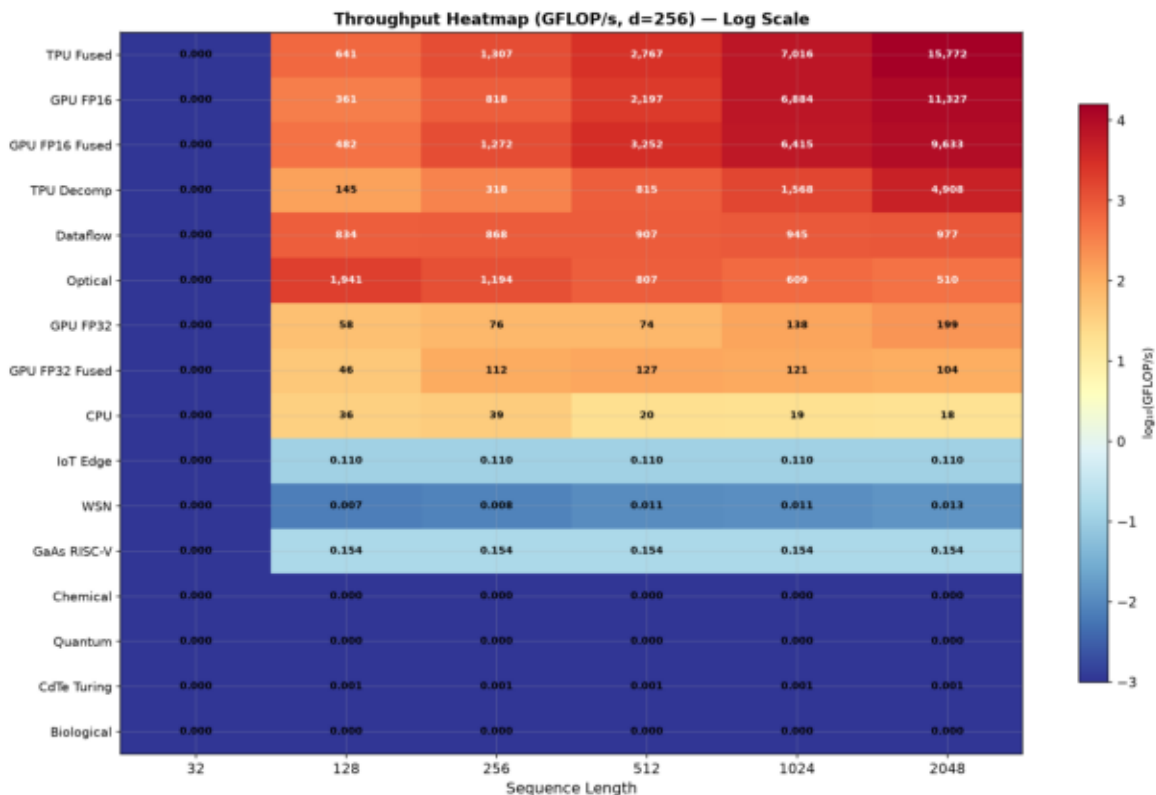


Figure 5: The heatmap shows unconventional paradigms at the bottom (dark blue)

11. Photonic Computing — Speed of Light Has an Interface Bottleneck

The optical MZI mesh paradigm provides a striking illustration of domain-crossing costs in mixed-signal architectures:

For matrix multiplication, the 64×64 Mach-Zehnder interferometer mesh performs analog multiply-accumulate at the speed of light. Each tile propagates in ~ 0.1 ns, with DAC/ADC conversion at 10 GHz. For the QK^T matmul at $n=2048$, $d=256$, this takes only ~ 0.003 seconds.

But softmax requires converting optical signals to electronic domain (O $\rightarrow$ E), computing exp/max/div electronically, then converting back (E $\rightarrow$ O). Each row incurs a 10 ns OEO latency plus electronic softmax computation at 2 GHz. For 2048 rows, this totals ~ 0.007 seconds — dominating execution time.

This is a direct extension of Conclusion 4 (Softmax as discriminator): the nonlinear operation that is 1% of FLOPs becomes the performance bottleneck on photonic hardware because it forces domain conversion.

Implication: all-optical softmax approximations (e.g., Kerr nonlinearities) could unlock the full potential of photonic accelerators.

12. Edge & Distributed Computing — When Communication Dominates

Two constrained-environment paradigms reveal fundamentally different performance bottlenecks:

IoT Edge (ARM Cortex-M4F at 168 MHz):

- 0.11 GFLOP/s — limited by clock speed and SRAM capacity (256 KB)
- 3 cycles per FP32 MAC (VMLA.F32), 10 cycles for LUT-based $\exp()$
- Matrices larger than $\sim 146 \times 146$ require tiling from flash $\rightarrow$ SRAM
- At $n=2048$, $d=256$: 49 seconds. Faster than GaAs (35s) per-clock, but lower absolute throughput due to 168 vs 1000 MHz clock.

WSN (64 distributed sensor nodes at 16 MHz):

- 0.013 GFLOP/s — wireless communication (250 kbps, IEEE 802.15.4) dominates over computation
- Each round: distribute tiles wirelessly, compute locally, collect results
- At $n=2048$, $d=256$: 404 seconds, of which $>95\%$ is communication overhead
- Softmax requires 3 global synchronization rounds (max, sum, broadcast)

Key insight: distributing attention across constrained nodes converts a compute-bound problem into a communication-bound one. The bottleneck shifts from FLOPs to bits-per-second.

Key Numbers for Quick Reference

At n=2048, d=256 (5.39 GFLOP of work):

Paradigm	GFLOP/s	Type	Key Detail
TPU Fused (JAX)	15,772	Measured	XLA compiler fusion, float32
GPU FP16 Decomposed	11,327	Measured	Tensor Cores, cuBLAS
GPU FP16 Fused SDPA	9,633	Measured	mem_efficient backend
TPU Decomposed (JAX)	4,908	Measured	Per-op JIT, float32
Dataflow (Maxeler)	977	Model	4 parallel pipes, 500 MHz
Optical (MZI Mesh)	510	Model	64x64 photonic, OEO bottleneck
GPU FP32 Decomposed	199	Measured	cuBLAS, no Tensor Cores
GPU FP32 Fused SDPA	104	Measured	math backend fallback
CPU Decomposed	18.1	Measured	AVX2, 4 cores
CPU Fused SDPA	17.8	Measured	No specialized backend
GaAs RISC-V	0.15	Simulated	1 GHz, scalar only
IoT Edge (ARM M4F)	0.11	Simulated	168 MHz, 3 cyc/MAC, 256KB SRAM
WSN (64 sensor nodes)	0.01	Simulated	16 MHz MCUs, 250 kbps wireless
CdTe Turing	0.0007	Simulated	10 MHz, no multiplier

At n=32, d=16 (capped due to extreme slowness):

Quantum (Gate-based)	0.0006	Simulated	1792 T-gates per multiply
Chemical (CRN)	0.0002	Simulated	1 kHz reactions, 256 wells
Biological (DNA)	$\sim 10^{-8}$	Simulated	10s per strand displacement

* TPU Decomposed uses simulation-estimated cycles (no JAX fused path)
Real TPU execution on Colab via JAX block_until_ready